Lab1：Datalab 实验报告

沈越

2026.10.9

1　实验结果

在终端中执行./check__ops.py bits.c 后的截图如下：

```
avinexsy@shenyue:/mnt/c/Users/25300/Desktop/TestLab_01$ python3 ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 5/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 16/16 operators
roundEvenPow2: 24/24 operators
midpointTowardFirst: 27/32 operators
isBetweenEitherOrder: 25/48 operators
mul5Sat: 26/30 operators
classifyAdd3: 33/52 operators
floatScaleThreeHalves: 41/60 operators
floatRoundEven: 24/65 operators
float_i2f: 25/40 operators
bitCount: 24/40 operators
bitReverse: 31/34 operators
All 19 functions passed operator checks.
```

在终端中执行./btest 后的截图如下：

```
avinexsy@shenyue: /mnt/c/Us
floatScaleThreeHalves: 41/60 operators
floatRoundEven: 24/65 operators
float_i2f: 25/40 operators
bitCount: 24/40 operators
bitReverse: 31/34 operators
All 19 functions passed operator checks.
avinexsy@shenyue:/mnt/c/Users/25300/Desktop/TestLab_01$ make clean && make
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
bits.c: In function 'classifyAdd3':
bits.c:378:7: warning: unused variable 'ovf1' [-Wunused-variable]
  378 |   int ovf1 = ((x ^ s1) & (y ^ s1)) >> 31;
      |       ^~~~
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
avinexsy@shenyue:/mnt/c/Users/25300/Desktop/TestLab_01$ ./btest
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
```

2 实验思路

P1: 构造只有最高位（第 31 位）为 1 的掩码 0x80000000。将 1 左移 31 位即可。

P2: 利用德·摩根定律实现异或 $x \oplus y = (x \wedge \neg y) \vee (\neg x \wedge y) = \neg(\neg(x \wedge \neg y) \wedge \neg(\neg x \wedge y))$。

P3: 提取符号位 sign = x » 31（若 $x < 0$ 为 -1 即全 1，若 $x \geq 0$ 为 0）。若 $x < 0$，则取补码相反数 ~x + 1；最后与 sign 按位与以保留/清零。

P4: 利用 src « 3 和 dst « 3 将字节编号转为位偏移量。先将 x 在 dst 处的字节清零（与 ~(0xFF « dst__shift) 取与），再提取 src 处的字节移位至 dst 位置（((x » src__shift) & 0xFF) « dst__shift），最后按位或合并。

P5: 先算术右移 x » n，再构造高 n 位全 0、低 32-n 位全 1 的掩码以消除符号位扩展。掩码利用 ~0 « (31 - n) « 1 构造。

P6: 分别提取高 4 位（x & 0xF0F0F0F0，通过掩码 0xF0 组合生成）和低 4 位（x & 0x0F0F0F0F），高 4 位右移 4 位、低 4 位左移 4 位后组合。

P7: ~x 将 0 转变为 1。第一个 lowest zero bit 对应的掩码可以通过 ~x & (~x + 1) 找到。将该位在 ~x 中消除（y = ~x & (~x - 1)，其中 ~x - 1 可写成 ~x + ~0），再对剩下的位提取最低的 1，即得到第二个 0 比特的位置掩码。

P8: 通过折叠（树状异或）计算 32 位的异或和。若 1 的个数为偶数，最终二进制最低位为 0，题目要求此时返回 1，因此最后取!(x & 1)。

P9: 将右移移出的低 n 位放到高位。偏移量为 32 __ n（即 32 + ~n + 1）。由于 $n = 0$ 时高位移位不能超过 31 位，需处理边缘情况或构造遮罩。

P10: 计算模 2^n 的余数 rem = x & ((1 « n) - 1) 以及一半门槛 half = 1 « (n - 1)。当 rem > half 或 (rem == half 且商为奇数) 时向上取整，否则向下截断。

P11: 避免加法溢出的中点计算公式为 $(x \wedge y) + ((x \oplus y) > 1)$。当 $(x + y)$ 为奇数（即 $x \oplus y$ 的最低位为 1）且 $x < y$（即 $x __ y < 0$）时，标准下取整结果偏向了 y，需调整 +1。

P12: 判断 x 是否介于 a 和 b 之间，等价于 $(x - a)$ 与 $(b - x)$ 同号。即 $(x - a) \times (b - x) \geq 0$。通过检查补码减法的符号位是否非负即可。

P13: 计算 $5x = (x < 2) + x$。溢出判定：检查 $4x$ 计算是否溢出（$(x < 2) > 2 \neq x$）以及与 $+x$ 相加时符号位是否异常。溢出时根据 x 的正负返回 INT_MAX (0x7FFFFFFF) 或 INT_MIN (0x80000000)。

P14: 将 $x + y + z$ 分步计算：先算 $s1 = x + y$，记录 $s1$ 相对 INT_MAX / INT_MIN 的溢出进位；再算 $s2 = s1 + z$，综合两个步骤的符号与溢出标志。

P15: 浮点数乘以 1.5。拆分符号位 s、阶码 exp 和尾数 frac。
1. 若 exp == 255（NaN 或 Inf），直接返回 uf。
2. 若 exp == 0（非规格化数），将 frac 乘以 3 并右移 1，注意 Round-to-nearest-even 舍入。
3. 规格化数（exp > 0）：加上偏置量，相当于尾数乘 1.5。

P16: 提取符号位 s (第 31 位)、阶码 exp (第 22 30 位) 和尾数 frac (第 0 ~22 位)。将无偏阶码计算为 $E = exp - 127$。分别讨论 $E < 0$(舍入 0 或 1.0），$E \geq 23$（原样返回 uf) 和 $0 \leq E < 23$（讨论小数部分）。

P17: 特殊值与符号处理 → 寻找最高有效位 → 对齐尾数与截断 → 向偶数舍入 → 组合结果

P18: 使用 parallel bit-counting 算法。构造掩码 0x55555555, 0x33333333, 0x0F0F0F0F，树状叠加每 2 位、4 位、8 位... 中 1 的个数。

P19: 使用分治法。两两交换 bit → 两两交换 2bit 块 → 4bit → 8bit →16bit，最后整体交换高低 16 位。